

# 第3章

## 指令系统与程序设计





## 3.2 C语言程序设计

---

- 3.1 指令系统与寻址方式
- 3.2 C语言程序设计



## 3.2.1 程序设计基础

---

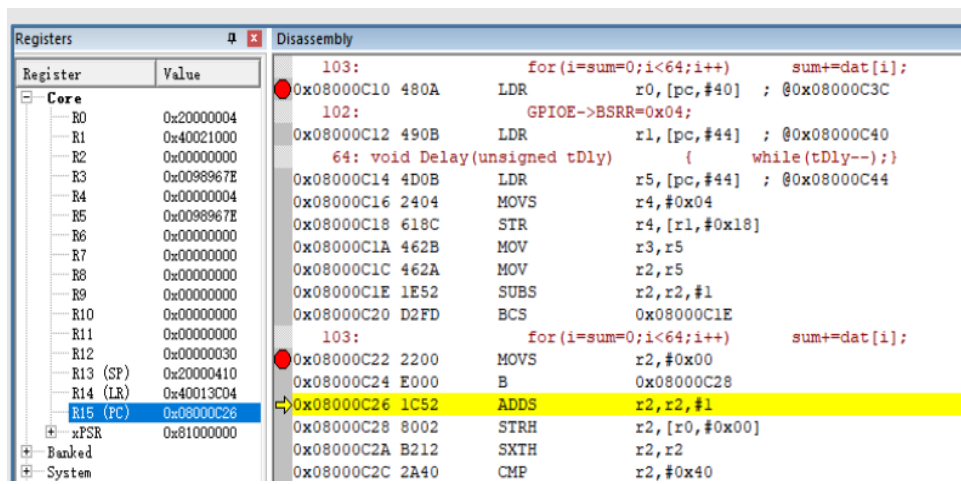
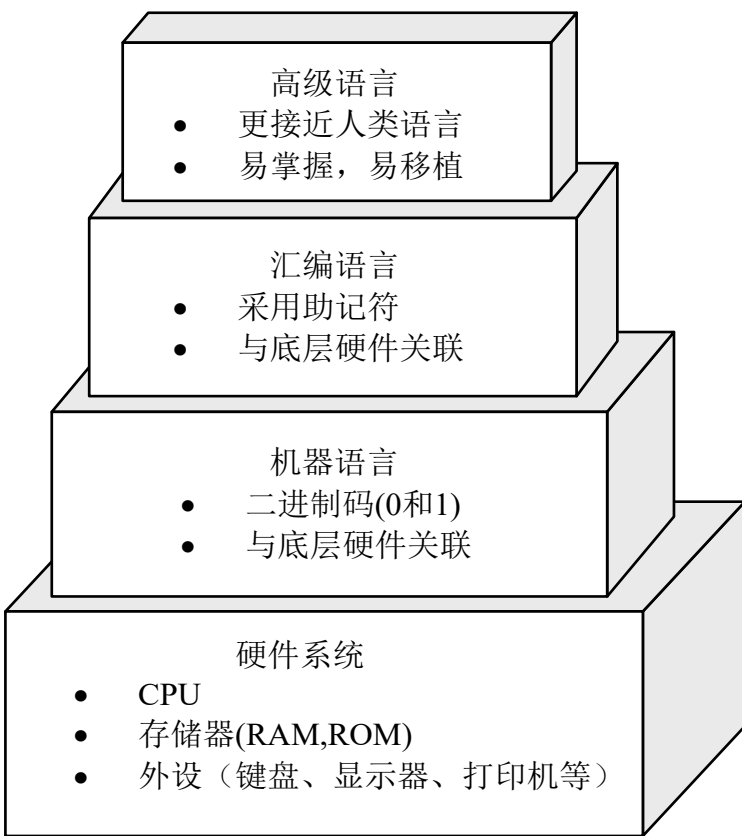
- 3.2.1 程序设计基础
- 3.2.2 C语言程序设计
- 3.2.3 MSP430集成开发环境



# 3.2.1 程序设计基础

软件需要借助于具体的语言来编写。编程语言，又称计算机语言，通常是一个能完整、准确和规则地表达人们的意图，并用以指挥或控制硬件系统工作的“符号系统”。

相对于最底层的硬件系统而言，编程语言有三个不同层次：机器语言，汇编语言和高级语言。



高级语言编写的程序需要被翻译成汇编语言，再由汇编程序汇编成机器语言才能执行。





## 3.2.1 程序设计基础

---

### (1) 机器语言

机器语言：用二进制代码表示的、CPU能直接识别和执行的一种机器指令的集合，具有灵活、直接执行和速度快等特点。

机器指令与CPU有着密切关系。通常，不同的CPU对应的机器指令也不同。但同一系列的CPU指令集常常是向下兼容的，如Intel80386指令包含了8086的指令集。



## 3.2.1 程序设计基础

---

例如：Intel8085CPU机器指令“00111110 00001001”，即3E 09H，是一条传送类指令，因为其操作码为3EH，它告诉机器完成一个向累加器ACC传送数据的操作。后一个字节09H则为操作数。因此，这条指令实现的是将立即数09H传送到累加器ACC中的操作。

显然，如果是8位数据宽度的程序存储器来存放该指令，则需要占据两个存储单元，其操作码和操作数会被分别存储在两个连续的存储单元当中，因此这是一条2字节的指令。不同的指令占用的存储单元数不同，例如，一个3字节指令则需要占据3个连续的存储空间。



## 3.2.1 程序设计基础

---

### (2) 汇编语言

人们利用与代码指令实际含义相近的英文缩写词、字母、数字等符号来取代指令代码编写程序，这就形成了常说的汇编语言，亦称为符号语言。

汇编语言是一种用助记符表示的仍然面向机器的计算机语言，其助记符与指令代码一一对应，基本保留了机器语言的灵活性。例如，前面提到的机器指令3E 09H所对应的汇编语言为MVI A, #09H，其中，MVI为助记符，表示“立即数传送”，A则指明了传送的目的地是累加器。显然，它虽与机器语言表示的含义相同，但直观性却提高很大。



## 3.2.1 程序设计基础

但是，机器语言是CPU可直接识别和执行的唯一语言，因此，用汇编语言编写的程序必须经过“汇编程序或汇编器(Assembler)”这个翻译软件的加工和处理，才会变成能够被CPU识别和处理的二进制代码程序。

通常，我们把用汇编语言等非机器语言书写好的符号程序称为**源程序**，而把翻译之后的机器语言程序称为**目标程序**。目标程序一经被保存在内存的预定位置上，就能被CPU处理和执行。



| 名称   | 助记符     | 机器码           |         | 说明                          |
|------|---------|---------------|---------|-----------------------------|
| 移动指令 | MOV A,n | 10110000<br>n | B0<br>n | 将立即数n放入累加器A中                |
| 加法指令 | ADD A,n | 00000100<br>n | 4<br>n  | 将累加器A中的数据与立即数n相加，结果放在累加器A中。 |
| 暂停   | HLT     | 11110100      | F4      | 停止所有操作                      |



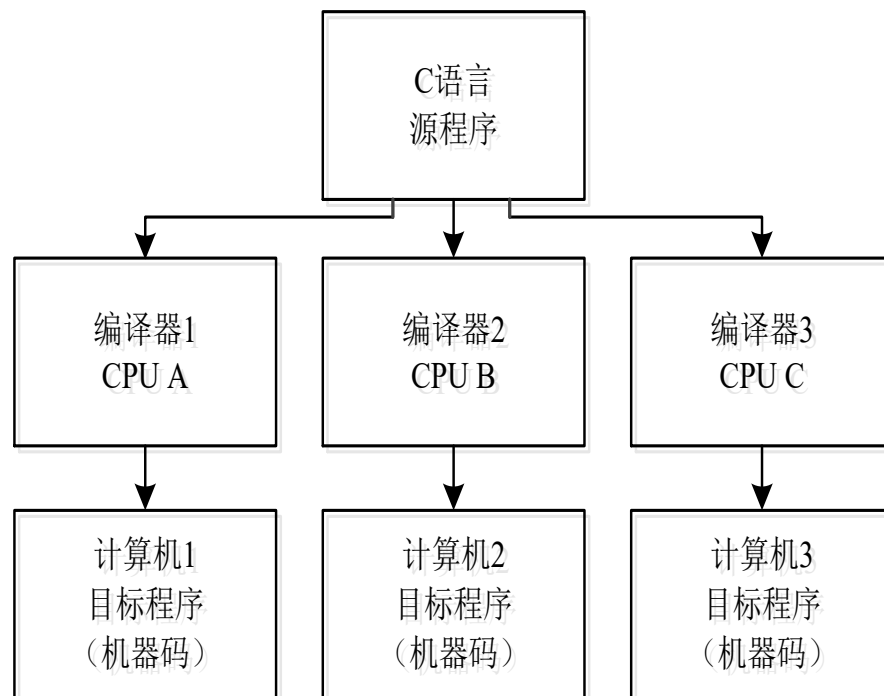
## 3.2.1 程序设计基础

### (3) 高级语言

高级语言是与我们的自然语言相近、并为计算机所接受和执行的编程语言，是面向用户而不是面向机器的语言。高级语言使我们编程时可以不顾及底层硬件的具体情况，给编写程序带来了极大的便利。

但是，高级语言同样无法被CPU直接识别和执行，也需要经过翻译转换成机器语言。无论何种机型的CPU，只要配备上相应的高级语言的“编译器（Compiler）或解释程序”，则用该高级语言编写的程序就可以通用。

如图所示，用C语言编写的源程序，在不同类型的CPU上运行时，只要通过相应的编译程序翻译成对应的机器代码，即目标程序即可运行。可见，**高级语言易于移植**。





## 3.2.1 程序设计基础

---

- 一般对于小程序来说，
  - ◆ 若是对硬件进行简单的控制可以用汇编语言，
  - ◆ 若更多涉及逻辑设计方面内容，则需要使用C语言。
- 对于稍大一些的程序来说，C语言的优势就十分明显了。就现代单片机程序设计来说，大多是以C语言为主，汇编语言为辅。即只有在那些对代码大小和效率要求较高的场合才使用汇编语言。



## 3.2.1 程序设计基础

---

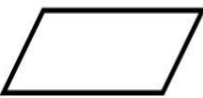
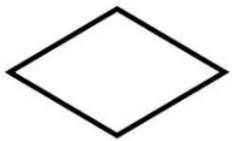
### 单片机程序设计的一般步骤

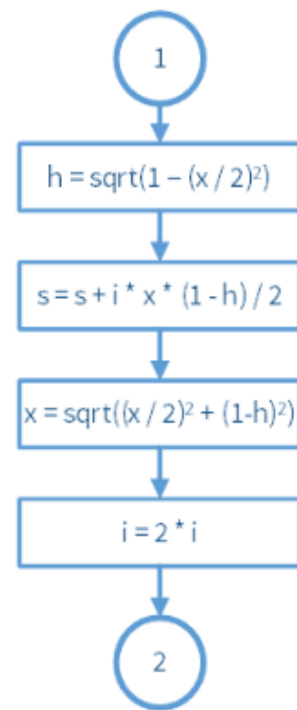
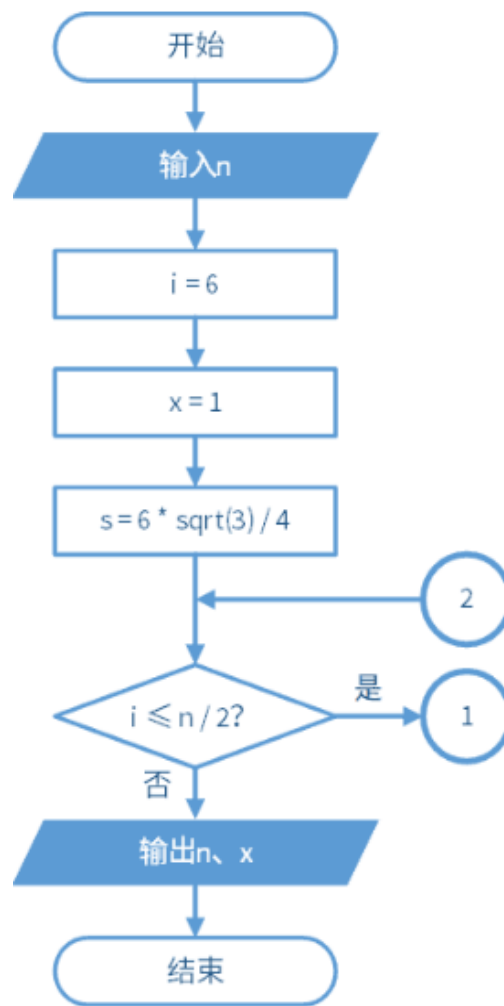
- (1) 需求分析、明确任务
- (2) 算法设计（计算机上解决问题的方法与步骤）
- (3) 芯片选择及合理分配单片机资源
- (4) 代码编写
- (5) 程序调试
- (6) 性能评估
- (7) 文档整理与编写

## 3.2.1 程序设计基础



### 程序流程图

| 图形符号                                                                                | 名称       | 功能                                         |
|-------------------------------------------------------------------------------------|----------|--------------------------------------------|
|     | 终端框(起止框) | 表示一个算法的起始和结束                               |
|     | 输入、输出框   | 表示一个算法输入和输出的信息                             |
|     | 处理框(执行框) | 赋值、计算                                      |
|    | 判断框      | 判断某一条件是否成立,成立时在出口处标明“是”或“Y”,不成立时标明“否”或“N”. |
|   | 流程线      | 连接程序框                                      |
|  | 连结点      | 连接程序框图的两部分                                 |







## 3.2.2 C语言程序设计

---

- 3.2.1 程序设计基础
- 3.2.2 C语言程序设计
- 3.2.3 MSP430集成开发环境



## 3.2.2 C语言程序设计

---

目前支持MSP430的C语言编译器很多，国内常用的是：IAR公司的IAR Embedded Workbench for MSP430 (EW430)；TI公司开发的Code Composer Studio (CCS)。

MSP430的C语言程序设计方法与标准C语言基本相同，但单片机与PC相比，资源匮乏。为更好地适应MSP430的程序设计，MSP430对标准C语言进行了扩展。主要表现在：  
数据类型和长度、关键字扩展及由此引起的函数扩展。

不同的430编译器对C的扩展不完全相同，但大多数情况下，源程序可以在各个版本的C430编译器上使用。



## 3.2.2 C语言程序设计

---

### 一、数据类型与运算符

#### 1. 标识符与关键字

- 标识符是指常量、变量、语句标号、数组、文件名以及用户自定义函数的名称。
- C语言规定标识符只能由字母、数字、下划线组成，并且只能由字母、下划线开头，所用**字母区分大小写**。
- C语言中一些已被赋予特定含义的标识符被称为关键字或保留字，**关键字不能用作标识符**。



## 3.2.2 C语言程序设计

---

### 2. 数据类型及存储长度

- C语言支持多种数据类型，根据各自特点可将其分为基本类型、构造类型、指针类型、空类型四大类。
- MSP430C语言的基本数据类型是由short(短整型、2B)、int(整型、2B)、long(长整型、4B)、char(字符型、1B)、float(浮点型、4B)、double(双精度型、4B)



## 3.2.2 C语言程序设计

---

- 整型有三种情况，分别由关键字short、int、long标识。整型又分为无符号整型和有符号整型，分别由关键字unsigned和signed标识。这两个可与前面三个关键字组合使用，如无符号整型记为unsigned int。
- 各种无符号类型所占内存字节数与相应的有符号类型相同。若无显式说明，系统一般默认是有符号类型。



## 3.2.2 C语言程序设计

---

- 整数常量的表示方式有十进制、八进制和十六进制三种表示方式。
  - ◆ 以0开头的数表示八进制数，
  - ◆ 以0x开头的数表示为十六进制数
  - ◆ 除此之外的数均是十进制数。
- 在单片机程序设计时，十六进制数和十进制数最为常用。



## 3.2.2 C语言程序设计

### 作业1

1. 列表给出以下数据类型的存储长度、有符号/无符号情况下分别可表示的十进制数的范围。

| 数据类型      | 存储长度 | 范围                   |
|-----------|------|----------------------|
| char      | 1B   | 无符号: [0 255]<br>有符号: |
| short     |      |                      |
| int       |      |                      |
| long      |      |                      |
| long long |      |                      |
| float     |      |                      |
| double    |      |                      |

2. 实验说明short与int型是否存在区别。

3. 以0开头的数表示八进制数, 这里0是“零”还是字母“0”? 给出实验说明和截图。



## 3.2.2 C语言程序设计

---

### 3. 运算符及优先级

- 运算符是告诉编译器执行特定算术或逻辑操作的符号。
- C语言把除了控制语句和输入输出以外的几乎所有基本操作都作为运算符处理。
- C语言运算符的运算范围很宽，可分为：**算术运算符、关系运算符、逻辑运算符、按位运算符、逗号运算符、复合运算符等。**





## 3.2.2 C语言程序设计

---

### 作业1

4. 列表给出c语言的所有运算符及优先级。

| 优先级 | 运算符 | 名称或含义 | 使用形式     | 说明    |
|-----|-----|-------|----------|-------|
| 7   | ==  | 等于    | 表达式==表达式 | 关系运算符 |
|     |     |       |          |       |



## 3.2.2 C语言程序设计

---

### 常见程序结构

结构化程序设计中，一个要求就是任何程序都由顺序、分支、循环三种基本结构进行构造。

#### 1. 顺序结构

顺序结构中的语句是按照书写的先后次序顺序执行的，每个语句都会被执行到，并且只能执行一次。

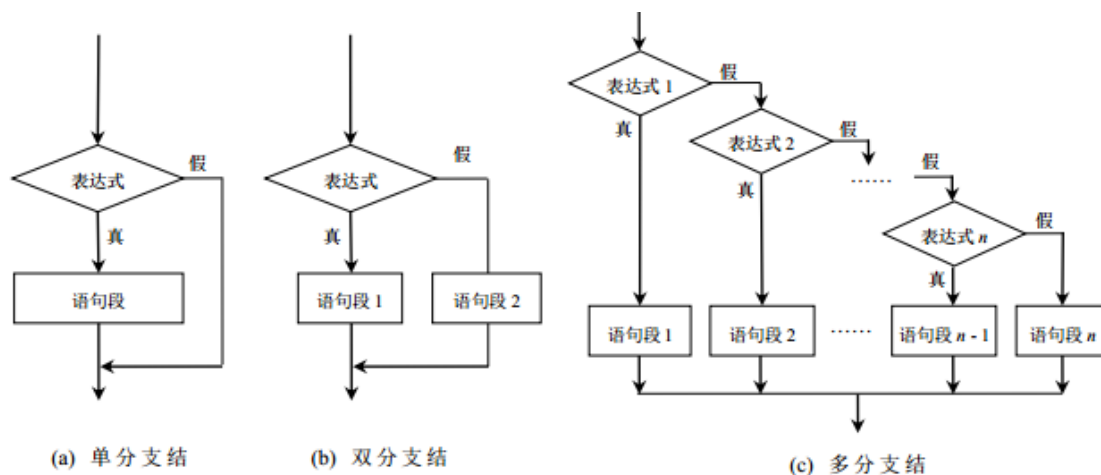
#### 2. 分支结构

分支结构程序又称为选择结构程序，是结构化程序设计的基本结构之一。

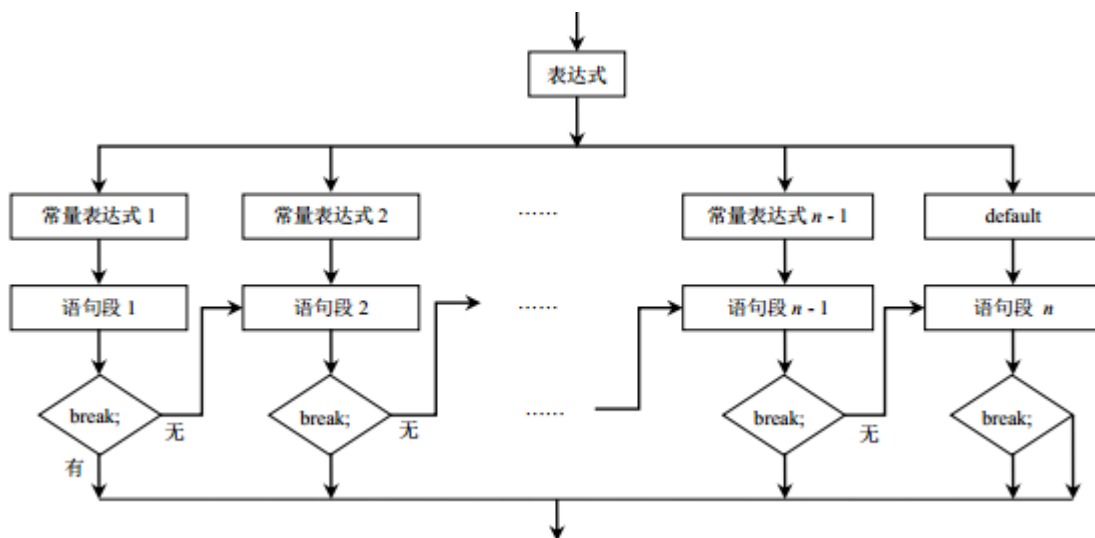


## 3.2.2 C语言程序设计

### ◆ (1) if 语句



### ◆ (2) switch 语句



## 3.2.2 C语言程序设计



### ● 3. 循环结构

- ◆ 在程序设计时有些语句往往需要重复执行数次，这时就需要使用循环结构。
- ◆ 循环结构在给定条件成立时将反复执行某程序段，直到条件不成立为止。

#### ◆ (1) **while** 语句

➤ **while (表达式) 循环体语句;**

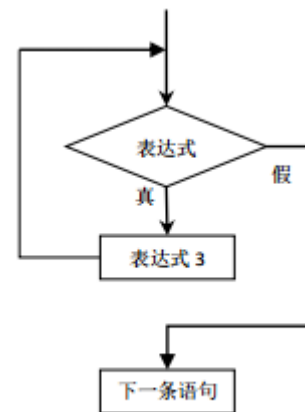


图 4.6 while 语句执行流程



## 3.2.2 C语言程序设计

### ● 3. 循环结构

#### ◆ (2) **do while** 语句

➤ **do {**

➤ 循环体语句

➤ **} while (表达式)**

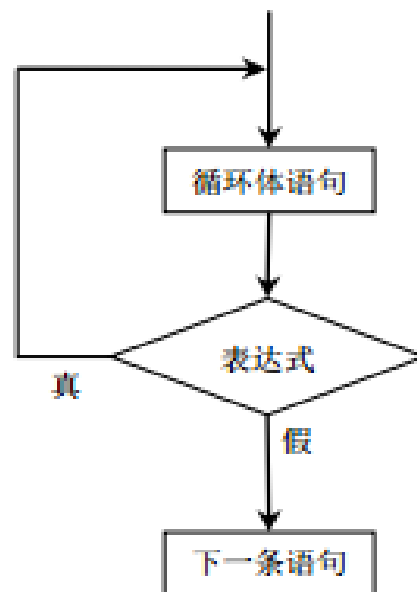


图 4.7 do while 语句执行流程



## 3.2.2 C语言程序设计

### ◆ (3) **for** 语句

➤ **for**(表达式 1; 表达式 2; 表达式 3)

➤ {循环体语句}

### ◆ (4) **goto** 语句

➤ **goto** 语句标号;

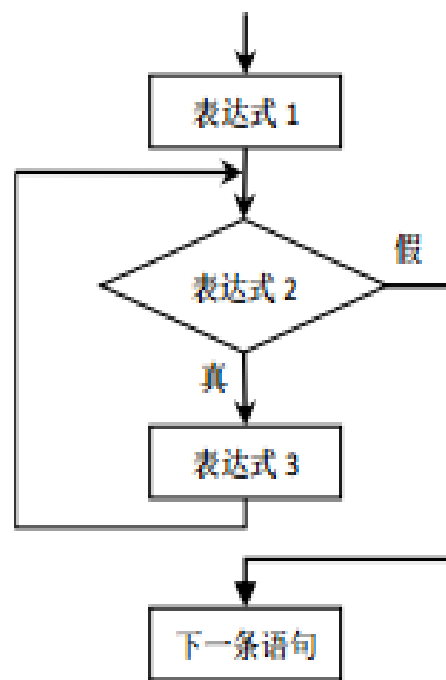


图 4.8 for 语句执行流程



## 3.2.2 C语言程序设计

### 数组

- ◆ 数组是由同一数据类型组成的集合体。
- ◆ 每个数据都有一个数组名。数组成员在内存中是连续存放的。数组名表示整个数组的起始地址即首地址。数据既可以是一维的，也可以是多维的。

|                    |                    |                    |                    |
|--------------------|--------------------|--------------------|--------------------|
| <del>a[0][0]</del> | <del>a[0][1]</del> | <del>a[0][2]</del> | <del>a[0][3]</del> |
| <del>a[1][0]</del> | <del>a[1][1]</del> | <del>a[1][2]</del> | <del>a[1][3]</del> |
| <del>a[2][0]</del> | <del>a[2][1]</del> | <del>a[2][2]</del> | <del>a[2][3]</del> |



## 3.2.2 C语言程序设计

### ◆ (1) 一维数组

➤ 数据类型 数组名[常量表达式];

### ◆ (2) 二维数组

➤ 数据类型 数组名[常量表达式 1][常量表达式 2];

|                    |         |         |                    |
|--------------------|---------|---------|--------------------|
| <del>a[0][0]</del> | a[0][1] | a[0][2] | <del>a[0][3]</del> |
| <del>a[1][0]</del> | a[1][1] | a[1][2] | <del>a[1][3]</del> |
| <del>a[2][0]</del> | a[2][1] | a[2][2] | <del>a[2][3]</del> |





## 3.2.2 C语言程序设计

---

### 函数

- 函数是 C 语言源程序的基本模块。
- C 程序全部由函数组成，并且必须有且只能有一个名为 main 的主函数。整个程序的执行从 main 函数开始，也在 main 函数中结束。



## 3.2.2 C语言程序设计

---

- 在 C 语言中，函数由函数头与函数体构成。函数定义的一般形式为：

函数类型 函数名([形参表])

{

变量定义;

功能语句;

返回语句;

}



## 3.2.2 C语言程序设计

---

- C语言使用 `return` 语句完成函数值的返回，即函数的返回值由 `return` 语句决定。`return` 语句的一般形式如下：
  - ◆ `return(表达式);` 或 `return 表达式;` 或 `return;`
  - ◆ 一旦执行`return`语句，函数就会返回到调用函数处。  
若无`return`语句，遇“`}`”时，自动返回调用函数。



## 3.2.2 C语言程序设计

---

### 1. 自定义函数与库函数

- ◆ 在 C 语言程序设计中，从程序编写者角度来看函数可分为自定义函数与标准函数两种。
- ◆ 自定义函数是指用户根据具体应用所编写的函数。
- ◆ 库函数是由系统或硬件提供商提供的标准化函数。



## 3.2.2 C语言程序设计

---

### ● 2. 函数调用

◆ 在 C 语言程序设计中，一个函数从编写完成到使用一般需要经历函数定义、函数声明以及函数调用三个过程。

◆ 函数声明的目的是告诉编译系统函数类型、参数个数及类型，在程序编译时以便对函数进行检查。自定义函数声明的方式一般形式为：

➤ 函数类型 函数名(形参表);



## 3.2.2 C语言程序设计

---

◆ 函数调用的一般形式为：

➤ 函数名(实参表);

◆ 被调函数在主调函数中有三种调用方式，第一种是将函数调用作为一个语句；第二种是调用的函数出现在表达式中，此时函数的返回值参与表达式的运算；第三种是将函数作为实参使用，函数的返回值即为实参的值。



## 3.2.2 C语言程序设计

---

### 3. 局部变量与全局变量

- ◆ 凡是在函数内部定义的变量，其只在本函数内有效，也就是说只能在本函数内才能使用它们，这些变量就称为**局部变量**。
- ◆ 在函数外部定义的变量即为外部变量，由于其有效范围从定义变量的位置开始到本源文件结束，所以又称为**全局变量**或全程变量。在有效范围内的函数均可以使用该变量。



## 3.2.2 C语言程序设计

---

- ◆全局变量增加了函数间的数据联系途径，合理利用全局变量可以使程序效率提高。
- ◆尽量少用全局变量，因为一直占用存储资源。
- ◆若全局变量与局部变量同名时，局部变量有效、全局变量无效，即全局变量被局部变量屏蔽掉了。





## 3.2.2 C语言程序设计

---

### 4. 变量的存储类型

- ◆ 从变量值存在的时间将变量分为**静态存储类型**和**动态存储类型**。全局变量属于静态存储类型，而像函数形参、大部分局部变量以及函数调用时需要的现场数据等均属于动态存储类型。
- ◆ 具体有 **auto**、**register**、**static** 和 **extern** 四种类型。



## 3.2.2 C语言程序设计

---

(1) **auto** 类型，又称为自动变量。函数中的局部变量，若不专门声明是 **static** 类型，则都是自动变量。

(2) **static** 类型，又称为静态变量。静态局部变量与局部变量的区别在于：在函数退出时，变量始终存在，保留原值。对于定义为 **static** 类型的全局变量，只在定义它的源文件中可见而在其它源文件中不可见。



## 3.2.2 C语言程序设计

---

(3) `extern` 类型，又称为外部变量。如果想要扩大全局变量的有效范围，就需要使用关键字 `extern` 来声明此全局变量。

(4) `register` 类型，又称为寄存器变量。寄存器变量是将用关键字声明的变量存放在 CPU 的寄存器中，由于寄存器数据传输速度快，这样可以提高运算速度。寄存器变量只能用于整型和字符型变量。同时寄存器变量的个数也是有限制的，具体视不同编译器而定。



## 3.2.2 C语言程序设计

---

### 指针类型

- 指针是 C 语言程序设计中一种重要的数据类型，也是 C 语言的精髓。
- 在 C 语言中地址称被为指针。

#### 1. 普通指针变量

◆ 指针变量的定义与一般变量的定义类似，定义的一般形式为：

**数据类型说明符 \*指针变量名；**

`int *data;    //data指向一个整型变量`

`static int *name;    //name是指向静态整型变量的指针变量`

`float *average;    // average是指向浮点型变量的指针变量`



## 3.2.2 C语言程序设计

---

C语言中变量的地址由编译系统分配，对用户完全透明，所以C语言提供了地址运算符&来提取变量的地址。

### 指针的赋值方法

#### 1) 指针变量定义时赋值

```
int x;
```

```
int *p = &x; //定义指针变量p，并将变量x的地址赋给p，即使p指向x
```

#### 2) 先定义后赋值

```
int x;
```

```
int *p; //定义指针变量p
```

```
p = &x; //将变量x的地址赋给p，即使p指向x
```



## 3.2.2 C语言程序设计

---

### 根据指针变量访问指针内容

```
int x = 0;    //定义变量x = 0
```

```
int *p = &x;  //定义指针变量p, 并将变量x的地址赋给p,即使p指向x  
*P = 15;
```

```
// x=?
```

```
// 修改x=15
```



## 3.2.2 C语言程序设计

---

### 2. 高级指针变量

指针变量不仅可以指向普通变量，还可以指向数组、函数等。

```
int a[8], *p;
```

```
p = a; //数组名表示数组的首地址，将其赋予指针变量，即使p指向数组a
```

```
int (*pf)(); //定义指向函数的指针变量pf
```

```
pf = fun; //使pf指向fun
```





## 3.2.2 C语言程序设计

### 预处理

C语言的一个重要功能，指在进行源程序编译之前所作的一些处理工作。

常见预处理指令

| 类型   | 预处理指令    | 含义                                               |
|------|----------|--------------------------------------------------|
| 宏    | #define  | 定义宏                                              |
|      | #undef   | 取消已定义的宏                                          |
| 文件包含 | #include | 包含一个源代码文件                                        |
| 条件编译 | #if      | 如果给定条件为真，则编译下面代码                                 |
|      | #ifdef   | 如果宏已经定义，则编译下面代码                                  |
|      | #ifndef  | 如果宏没有定义，则编译下面代码                                  |
|      | #elif    | 如果前面的#if 给定条件不为真，当前条件为真，则编译下面代码，其实就是 else if 的简写 |
|      | #endif   | 结束一个#if.....#else 条件编译块                          |





## 3.2.2 C语言程序设计

### 1. 宏

在 C 语言源程序中允许用一个标识符来表示一个字符串，称为“宏”。

◆ (1) 无参数：指宏名后面不含有参数，其定义的一般形式为：

**#define 宏名 字符串**      字符串可以是常数、表达式、格式串等

例如      **#define PI 3.1415926** //定义宏PI为圆周率

**#define S (PI\*r\*r)**    //定义宏S为圆的面积

◆ (2) 带参数宏：指宏名后面含有参数，其定义的一般形式为：

**#define 宏名(形参表) 字符串**    其中，字符串中含有各个形参

例如      **#define MAX(a,b) (a>b)? a:b**    //功能是返回a、b中的最大值



## 3.2.2 C语言程序设计

---

### 2. 文件包含

- ◆ 一个大的程序可由多个模块组成，每个模块可由不同的人开发。此时可将公用的符号常量或宏定义等单独组成一个公用文件。
- ◆ 当需要在文件中使用这些公用的符号常量或宏定义时，可在文件的开头使用包含指令包含该公用文件即可使用。
- ◆ 文件包含指令的一般形式为：

**#include "文件名"或<文件名>**

**#include "msp430g2553.h"**

**#include "msp430f2618.h"**



# 3.2.2 C语言程序设计

## 3. 条件编译

条件编译可按不同的条件来编译程序的不同部分，进而产生不同的目标代码文件。条件编译十分有利于程序的移植和调试。

条件编译的三种使用方式

|     | 第一种形式                                                                                   | 第二种形式                                                                                    | 第三种形式                                                                                 |
|-----|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 写格式 | <code>#ifdef</code> 宏标识符<br>程序段 1<br><code>#else</code><br>程序段 2<br><code>#endif</code> | <code>#ifndef</code> 宏标识符<br>程序段 1<br><code>#else</code><br>程序段 2<br><code>#endif</code> | <code>#if</code> 常量表达式<br>程序段 1<br><code>#else</code><br>程序段 2<br><code>#endif</code> |
| 能   | 如果宏标识符已被定义则对程序段 1 进行编译；否则对程序段 2 进行编译。<br>如果没有程序段 2(它为 空),本格式中的 <code>#else</code> 可以没有  | 如果宏标识符未被定义则对程序段 1 进行编译，否则对程序段 2 进行编译。<br>这与第一种形式的功能正相反                                   | 如常量表达式的值为真(非 0)，则对程序段 1 进行编译，否则对程序段 2 进行编译。<br>因此可以使程序在不同条件下，完成不同的功能                  |



## 3.2.2 C语言程序设计

---

### typedef 定义类型

- C语言中允许用户为数据类型取“别名”，即可对现有数据类型的名字进行重新命名。
- 通过关键词 typedef 来实现，其语法格式为：

**typedef type name;**

如：typedef unsigned int COUNT; //为基本数据类型定义新的类型名



## 3.2.2 C语言程序设计

---

### 规范化编程

规范化编程由以下几方面入手

#### 1. 变量命名规范

- ◆ 变量命名时尽量使用具有说明性的名字做到**见名知义**。
- ◆ 变量命名要清晰、明了、有明确含义。
- ◆ 一般变量名使用完整的单词或大家基本可以理解的缩写，避免使人产生误解。对变量的定义，尽量位于函数的开始位置。



## 3.2.2 C语言程序设计

---

### 2. 函数命名规范

- ◆ 函数的命名应该尽量用英文表达出函数完成的功能。
- ◆ 遵循“功能模块名\_动宾结构”的命名法则，函数名中动词在前，并在命名前加入函数的前缀，函数名的长度不得少于8个字母。例如 **Key\_GetKey()**、**Sys\_Init()**。



## 3.2.2 C语言程序设计

---

### 3. 代码书写

- ◆ 采用缩进风格;
- ◆ 加空行;
- ◆ 多个短句不允许放在同一行;
- ◆ if、for、do、while、case、switch、default 等语句自占一行, 且 if、for、do、while 等语句的执行语句部分无论多少都要加括号{}。
- ◆ 括号(程序块的分界符)也独占一行;
- ◆ 较长的语句应该换行。



## 建议的风格

## 不建议的风格



```
void Function(int x)
{
    ... // program code
}
```

```
void Function(int x){
... // program code
}
```

```
if (condition)
{
    ... // program code
}
else
{
    ... // program code
}
```

```
if (condition){
... // program code
}
else {
... // program code
}
```

```
for (initialization; condition; update)
{
    ... // program code
}
```

```
for (initialization; condition; update){
... // program code
}
```

```
while (condition)
{
    ... // program code
}
```

```
while (condition){
... // program code
}
```

如果出现嵌套的 { } ， 则使用缩进对齐， 如

```
⋮
{
    ...
    {
        ...
    }
    ...
}
```





## 3.2.2 C语言程序设计

---

### 4. 计算精度

单片机程序设计时不要盲目追求高精度，应根据单片机性能、RAM 资源和实际应用情况确定是否采用浮点计算。

### 5. 防止数据溢出

- ① 准确掌握不同数据类型对应的数据范围。
- ② 避免马虎粗心致使数据溢出。
- ③ 避免使用数据类型间的隐式转换，通常的做法是用强制类型转换替代隐式转换。



## 3.2.2 C语言程序设计

---

### 5. 编写注释

- ◆ 写注释时尽量少使用缩写，尤其是不常用的缩写。
- ◆ 每个文件的头部都应该编写注释用于说明本文件的重要信息，如版权，版本号，生成日期，作者，内容，功能，修改日志等。

#### ● 在哪些地方写注释？

- ◆ 在用户自定义函数前
  - 对函数接口进行说明
  - 函数功能 + 入口参数 + 出口参数 + 返回值（包括出错处理）



## 3.2.2 C语言程序设计

---

- ◆ 在一些重要的语句块上方
  - 对代码的功能、原理进行解释说明
- ◆ 在一些重要的语句行右方
  - 定义一些非通用的变量
  - 函数调用
  - 较长的、多重嵌套的语句块结束处
- ◆ 在修改的代码行旁边加注释



## 3.2.2 C语言程序设计

---

- 可灵活运用的一些规则

- ◆ 注释可长可短，但应画龙点睛，重点加在语义转折处

- ◆ 简单的函数可以用一句话简单说明

//两数交换

```
void Swap(int *x, int *y)
```

- ◆ 内部使用的函数可以简单注释，供别人使用的函数必须严格注释，特别是入口参数和出口参数



## 3.2.2 C语言程序设计

---

- 写注释时的注意事项

- ◆ 注释不是白话文翻译，不要鹦鹉学舌

- *Don't write comments that repeat the code*

- ◆ 不写做了什么，写想做什么

- *Do write illuminating comments that explain approach and rationale*

- ◆ 边写代码边注释

- ◆ 修改代码同时修改注释



## 3. 2. 3 MSP430集成开发环境

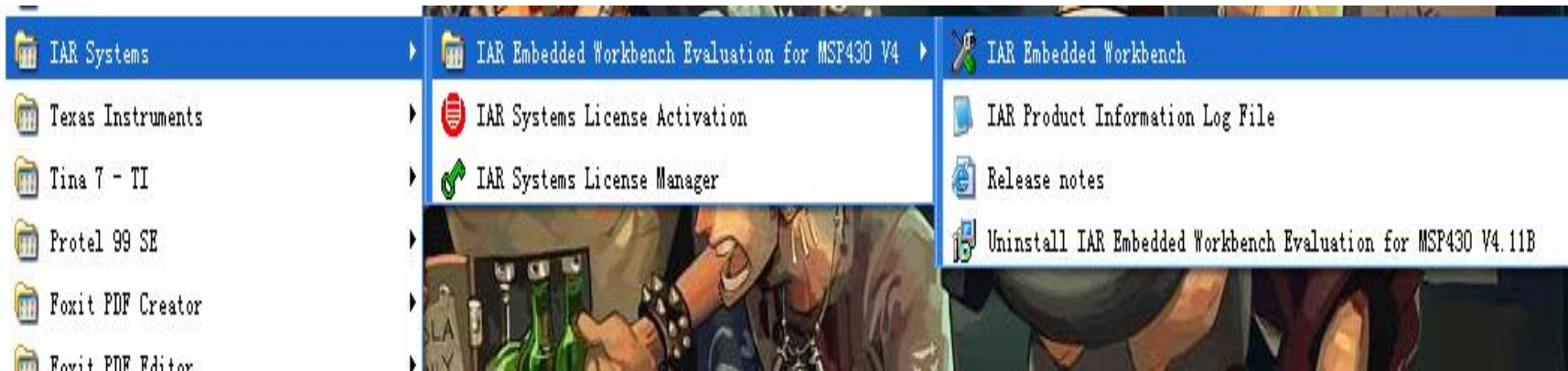
---

- 3. 2. 1 程序设计基础
- 3. 2. 2 C语言程序设计
- 3. 2. 3 MSP430集成开发环境

## ◆编程环境—IAR, CCS

### ➤编程环境—IAR

IAR Systems是全球领先的嵌入式系统开发工具和服务的供应商。公司成立于1983年，迄今已有27年，提供的产品和服务涉及到嵌入式系统的设计、开发和测试的每一个阶段，包括：带有C/C++编译器和调试器的集成开发环境(IDE)、实时操作系统和中间件、开发套件、硬件仿真器以及状态机建模工具。



## ➤编程环境—CCS

CCS（Code Composer Studio）是TI公司研发的一款具有环境配置、源文件编辑、程序调试、跟踪和分析等功能的集成开发环境，能够帮助用户在一个软件环境下完成编辑、编译、链接、调试和数据分析等工作。CCS是MSP430软件开发的理想工具。

**作业：** 下载并安装CCS



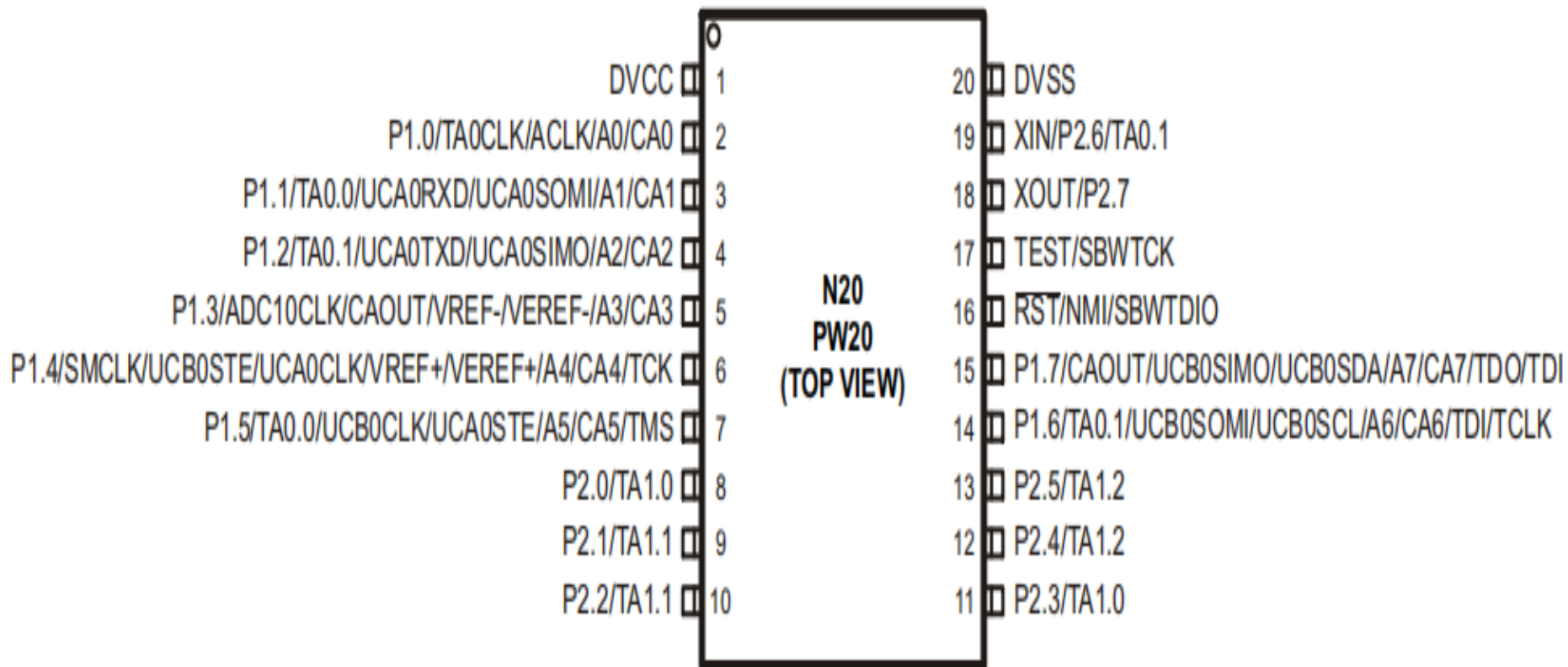
# ◆MSP430G2553单片机概述

---

## G2553主要特点:

- 低电压供电: 1.8V-3.6V
- 16MHz主频, 5种低功耗模式
- 8输入10Bit 200ksps (sample-per-second) ADC
- 两个16Bit TimerA
- USCI(通用串行通信接口)接口, 支持Uart(通用异步收发), IrDA (红外数据组织, Infrared Data Association, 红外线接口), SPI 和I<sup>2</sup>C功能。
- 通用型模拟比较器
- 16KB FLASH、512B RAM

# ◆ MSP430G2553单片机概述



# ◆MSP430G2553单片机概述

---

## MSP430G2553内部资源

- 三种可选择系统时钟 (**ACLK、MCLK、SMCLK**)
- 通用**I/O(GPIO)**
  - 可独立编程
  - 可提供输入、输出与中断（边沿可选）的任意组合
  - 所有寻址指令可对端口控制寄存器进行读/写访问
  - 每个 **I/O** 具有一个可独立编程的上拉/下拉电阻
  - 某些器件/引脚具有触摸按键模块 (**PinOsc**)

# ◆MSP430G2553单片机概述

- **16 位 Timer\_A2**

- **2 个捕获/比较寄存器**（捕获：利用外部信号的上升沿、下降沿或上升下降沿触发来测量外部或内部事件，如利用两次捕获的值来测量脉冲的宽度）
- 丰富的中断功能

- **串行通信**

- 支持 **I<sup>2</sup>C**、**SPI** 以及 **UART** 的 **USCI**

- **Comparator\_A+**

- 可设定反相和同相输入
- 可选的 **RC** 输出滤波器
- 可直接输出至 **Timer\_A2** 捕获输入
- 具有中断能力

# ◆MSP430G2553单片机概述

- **8 通道/10 位 200 kbps SAR ADC**
  - 8 个外部通道（取决于器件）
  - 内置电压和温度传感器
  - 可编程的参考电压
  - **DTC**：转换结果传送控制，与**DMA**类似，可在无需 **CPU** 干预的情况下将结果发送至存储器。这种方式转换速度快，适用于高速采样模式。
  - 具有中断能力
- **欠压复位(Brownout reset)**
  - 可在上电和断电期间提供正确的复位信号
- **WDT+ 看门狗定时器**
  - 也可用作一个普通定时器

# ◆MSP430G2553的开发

---

**LaunchPad** 是一款适用于TI的MSP430G2xx 系列产品的完整开发解决方案，可支持多达20 个引脚，提供板上Flash 仿真工具，以直接连接至PC 轻松进行编程、调试和评估。

# ◆MSP430G2553的开发



| 器件                | 引导加载器 (BSL) | 嵌入式仿真模块 (EEM) | 闪存 (KB) | RAM (B) | Timer_A | COMP_A+ 通道 | 10 通道 ADC | USCI A0/B0 | 时钟           | I/O | 封装类型           |
|-------------------|-------------|---------------|---------|---------|---------|------------|-----------|------------|--------------|-----|----------------|
| MSP430G2553IRHB32 | 1           | 1             | 16      | 512     | 2x TA3  | 8          | 8         | 1          | LF, DCO, VLO | 24  | 32 引脚 QFN 封装   |
| MSP430G2553IPW28  |             |               |         |         |         |            |           |            |              | 24  | 28 引脚 TSSOP 封装 |
| MSP430G2553IPW20  |             |               |         |         |         |            |           |            |              | 16  | 20 引脚 TSSOP 封装 |
| MSP430G2553IN20   |             |               |         |         |         |            |           |            |              | 16  | 20 引脚 PDIP 封装  |

# LaunchPad with MSP430G2553

Revision 1.5

Flash 16 KB  
Serial Hardware

|         |          |    |      |    |
|---------|----------|----|------|----|
| +3.3V   |          |    |      | 1  |
| RED_LED |          | A0 | P1_0 | 2  |
|         | RXD      | A1 | P1_1 | 3  |
|         | TXD      | A2 | P1_2 | 4  |
| PUSH2   |          | A3 | P1_3 | 5  |
|         |          | A4 | P1_4 | 6  |
|         | SCK (B0) | A5 | P1_5 | 7  |
|         | CS (B0)  |    | P2_0 | 8  |
|         |          |    | P2_1 | 9  |
|         |          |    | P2_2 | 10 |



Hardware  
Pin number

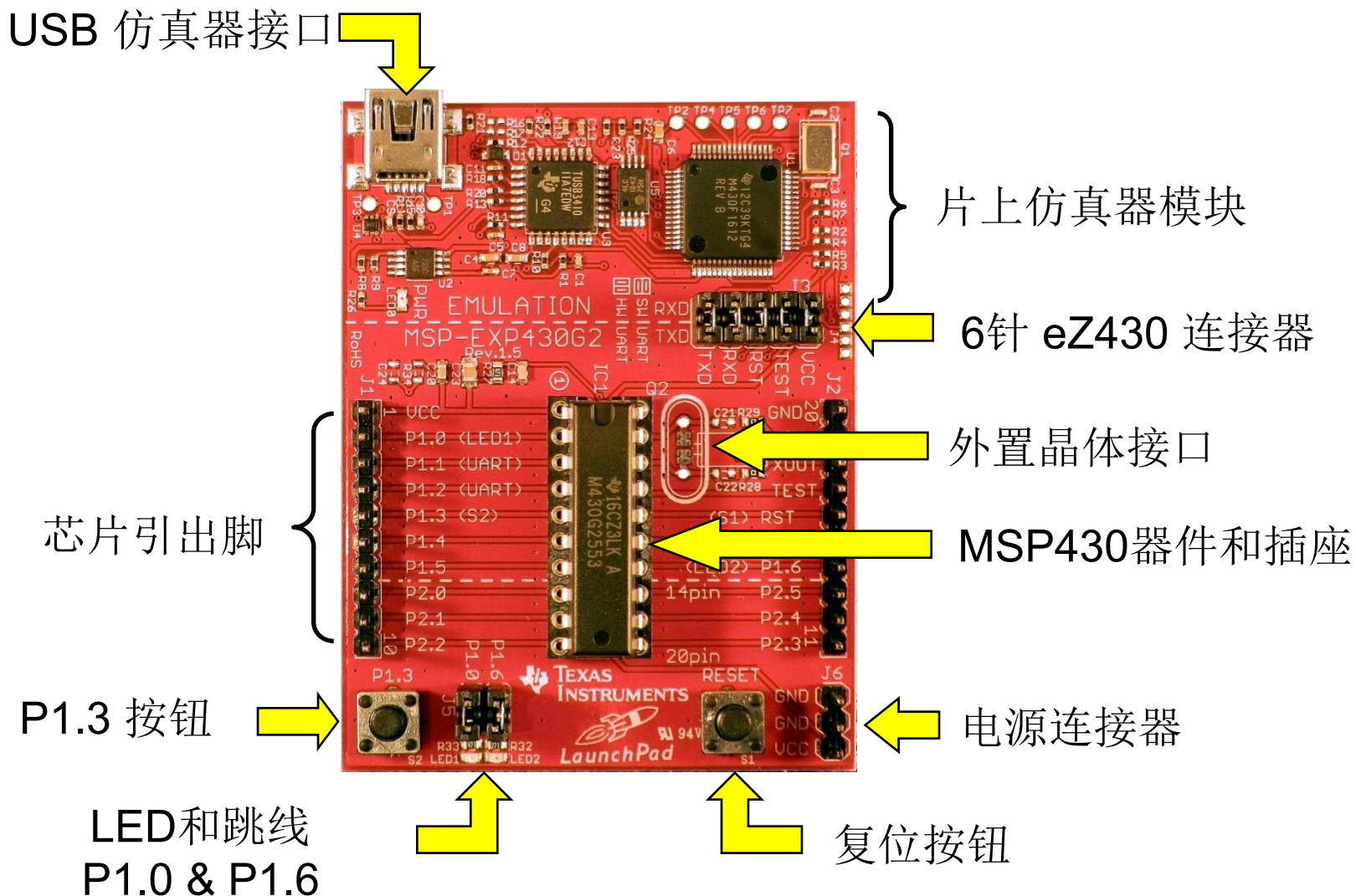
IO  
Serial UART  
SPI

analogRead()  
digitalRead() and digitalWrite()  
digitalRead(), digitalWrite()  
and analogWrite()

|    |      |    |     |           |
|----|------|----|-----|-----------|
| 20 |      |    |     | GROUND    |
| 19 | P2_6 |    |     | XIN       |
| 18 | P2_7 |    |     | XOUT      |
| 17 |      |    |     | TEST      |
| 16 |      |    |     | RESET     |
| 15 | P1_7 | A7 | SDA | MOSI (B0) |
| 14 | P1_6 | A6 | SCL | MISO (B0) |
| 13 | P2_5 |    |     | GREEN_LED |
| 12 | P2_4 |    |     |           |
| 11 | P2_3 |    |     |           |



USB 仿真器接口



**MSP430G2553 launchpad各部分结构**

# 仿真器的作用:

单片机在软件开发的过程中需要对软件进行调试，观察其中间结果，排除软件中存在的问题。但是由于单片机的应用场合问题，其不具备标准的输入输出装置，受存储空间限制，也难以容纳用于调试程序的专用软件，因此要对单片机软件进行调试，就必须使用单片机仿真器。

单片机仿真器具有基本的输入输出装置，具备支持程序调试的软件，使得单片机开发人员可以通过单片机仿真器输入和修改程序，观察程序运行结果与中间值，同时对与单片机配套的硬件进行检测与观察，可以大大提高单片机的编程效率和效果。

# MSP430开发流程

